ASTRAFORGE AEROSPACE · YNOV BACHELOR 2

ORBITAL COMMAND

Jour 2 — React Flight Deck

DYLAN · JEUDI 3 SEPTEMBRE 2026 · 4 HEURES · 20 % THÉORIE · 80 % PRATIQUE

BUTS DU COURS

- Découper une interface en composants
- Définir des contrats de props
- Gérer l'état local avec useState
- Filtrer et rechercher des véhicules
- Habiller le cockpit avec Tailwind CSS

SÉQUENCE 1 · BRIEFING

REACT FLIGHT DECK

Jour 2 — Transformer la flotte typée en interface interactive

Les modèles sont prêts. Il est temps de construire le cockpit.

SÉQUENCE 1 · BRIEFING

Hier, nous avons sécurisé les données

TypeScript nous a donné les fondations du centre de contrôle

- types et interfaces
- unions discriminantes
- narrowing et exhaustivité
- génériques
- projet Vite React-TypeScript
- premier composant `VehicleCard`

Qu'est-ce que TypeScript a déjà empêché dans votre projet ?

SÉQUENCE 1 · BRIEFING

Aujourd'hui, nous construisons le registre de flotte

Une interface composée de petites unités indépendantes et réutilisables

- découper une interface monolithique
- définir des contrats de props
- composer des composants avec `children`
- gérer l'état local avec `useState`
- filtrer et rechercher des véhicules
- construire une interface responsive avec Tailwind CSS

SÉQUENCE 1 · BRIEFING

Plan de vol

Cinq checkpoints avant l'ouverture du Flight Deck

1. Découper l'interface — composants, responsabilités et arbre React
2. Faire circuler les données — props, valeurs par défaut et composition
3. Rendre l'interface vivante — `useState`, immutabilité, listes et conditions
4. Réagir aux opérateurs — événements, recherche, filtres et état dérivé
5. Habiller le cockpit — Tailwind CSS, responsive et dark mode

4 heures — 5 micro-TP · 80 % de pratique

SÉQUENCE 1 · BRIEFING

Un registre de flotte interactif

La console doit présenter, filtrer et mettre à jour les véhicules AstraForge

- chaque véhicule possède une carte
- les statuts sont immédiatement identifiables
- la flotte peut être filtrée et recherchée
- un véhicule peut passer en maintenance
- l'interface fonctionne sur mobile et bureau
- le dark mode est amorcé

SÉQUENCE 2 · DÉCOUPER L'INTERFACE

01 — Découper l'interface

Un cockpit complexe reste un assemblage de composants simples

TP 1 — Monolith Separation

Tout mettre dans App.tsx fonctionne… au début

Chaque nouvelle fonctionnalité augmente la difficulté de modification

app-monolith.tsx

```tsx
function App() {
  return (
    <main>
      <header>...</header>

      {vehicles.map((vehicle) => (
        <article>
          <h2>{vehicle.name}</h2>
          <p>{vehicle.status}</p>
          <p>{vehicle.model}</p>
        </article>
      ))}
    </main>
  )
}
```

Combien de responsabilités différentes voyez-vous ici ?

Un composant transforme des données en interface

Il reçoit des informations et retourne une description de l'écran

status-badge.tsx

```tsx
function StatusBadge() {
  return <span>Ready</span>
}
```

- un nom de composant commence par une majuscule
- un composant retourne du JSX
- un composant peut utiliser d'autres composants
- son rendu dépend de ses props et de son état

SÉQUENCE 2 · DÉCOUPER L'INTERFACE

L'interface est un arbre de composants

Chaque niveau possède une responsabilité identifiable

- `App`
- `FleetPage`
- `FleetToolbar` — `SearchField`, `StatusFilter`
- `FleetGrid`
- `VehicleCard` — `StatusBadge`, `Metric`

Les données descendent dans l'arbre grâce aux props.

Un composant doit avoir une raison claire de changer

Le découpage suit les responsabilités, pas chaque balise HTML

Créer un composant lorsqu'un bloc :

- possède une logique identifiable
- doit être réutilisé
- devient difficile à lire
- évolue indépendamment
- représente un concept du produit

À éviter : `DivComponent`, `TitleWrapper`, `TextContainer`, `BlueBorderBox`

TP 1 — Monolith Separation

30 minutes · Découpez la console sans modifier son apparence

Créer au minimum :

- `FleetGrid`
- `VehicleCard`
- `StatusBadge`
- `Metric`

1. Identifier les responsabilités
2. Déplacer le JSX
3. Faire circuler les données
4. Conserver le rendu existant
5. Vérifier les types

Aucun composant ne doit relire directement le tableau global.

Le monolithe est détruit si…

Le comportement reste identique, mais la structure est plus lisible

- quatre composants minimum
- props typées
- aucun copier-coller de carte
- aucun accès global caché
- aucune erreur TypeScript
- même résultat visuel

Bonus : ajouter un nouveau composant sans toucher à `App.tsx`.

SÉQUENCE 2 · DÉBRIEF

Plus de composants ne signifie pas toujours plus de clarté

Le découpage doit rendre le changement plus simple

1. Le nom explique-t-il la responsabilité ?
2. Le composant peut-il être compris isolément ?
3. Reçoit-il trop de données ?
4. Existe-t-il seulement pour déplacer trois lignes ?
5. Où placeriez-vous une future interaction ?

SÉQUENCE 3 · PROPS ET COMPOSITION

02 — Faire circuler les données

Les props constituent le contrat public du composant

TP 2 — Control Panel Slots

Un composant déclare ce dont il a besoin

TypeScript vérifie chaque utilisation du composant

status-badge-props.tsx

```tsx
interface StatusBadgeProps {
  status: VehicleStatus
}

function StatusBadge({ status }: StatusBadgeProps) {
  return <span>{status}</span>
}

<StatusBadge status={vehicle.status} />
```

Les props sont un contrat : TypeScript refuse une utilisation incomplète.

Une prop optionnelle doit avoir un comportement prévisible

Les valeurs par défaut réduisent les conditions dispersées

metric-props.tsx

```tsx
interface MetricProps {
  label: string
  value: string | number
  unit?: string
}

function Metric({
  label,
  value,
  unit = "",
}: MetricProps) {
  return <p>{value} {unit}</p>
}
```

Dans quel cas `unit` doit-elle réellement être obligatoire ?

Une prop doit refléter le besoin du composant

Un composant trop informé devient difficile à réutiliser

status-badge-scope.tsx

```tsx
// à éviter
<StatusBadge vehicle={vehicle} />

// à privilégier
<StatusBadge status={vehicle.status} />
```

Transmettre la donnée minimale qui permet au composant de remplir sa mission.

children crée un emplacement pour le contenu

La structure reste stable, le contenu devient flexible

panel-children.tsx

```tsx
interface PanelProps {
  title: string
  children: React.ReactNode
}

function Panel({ title, children }: PanelProps) {
  return (
    <section>
      <h2>{title}</h2>
      {children}
    </section>
  )
}

<Panel title="Fleet Status">
  <FleetGrid vehicles={vehicles} />
</Panel>
```

`Panel` organise la structure sans connaître le contenu métier.

Préférez la composition aux props qui pilotent tout

Un composant ne doit pas devenir un moteur de templates

panel-composition.tsx

```tsx
// à éviter
<Panel
  showHeader
  showFooter
  compact
  dark
  centered
  withWarning
/>

// à privilégier
<Panel>
  <PanelHeader />
  <FleetGrid />
  <AlertSummary />
</Panel>
```

TP 2 — Control Panel Slots

34 minutes · Construisez une composition réutilisable

Créer :

- `Panel`
- `PanelHeader`
- `MetricGrid`
- `Metric`

Puis composer un panneau de flotte, un panneau de disponibilité et un panneau de maintenance.

SÉQUENCE 3 · DÉBRIEF

La composition sépare la structure du contenu

Le composant parent organise sans connaître tous ses usages futurs

À comparer entre groupes :

- nombre de props de `Panel`
- niveau de réutilisation
- responsabilité de `PanelHeader`
- emplacement des styles
- lisibilité du JSX d'utilisation

SÉQUENCE 4 · ÉTAT LOCAL

03 — Rendre l'interface vivante

L'état mémorise les informations qui évoluent à l'écran

TP 3 — Ground or Ready?

Modifier une variable ne demande pas un nouveau rendu

React doit être informé qu'une donnée visible a changé

local-variable.tsx

```tsx
let selectedStatus = "ready"

function changeStatus() {
  selectedStatus = "maintenance"
}
```

La variable change, mais React ne recalcule pas nécessairement l'interface.

useState mémorise une valeur entre deux rendus

Le setter demande à React de recalculer le composant

usestate.tsx

```tsx
const [selectedStatus, setSelectedStatus] =
  useState<VehicleStatus | "all">("all")

setSelectedStatus("maintenance")
```

`selectedStatus` – valeur actuelle · `setSelectedStatus` – mise à jour · `"all"` – valeur initiale

Chaque rendu voit un instantané de l'état

Une mise à jour prépare un nouveau rendu, elle ne réécrit pas l'ancien

functional-update.tsx

```tsx
function increment() {
  setCount((current) => current + 1)
}
```

Utiliser la forme fonctionnelle lorsque la nouvelle valeur dépend de l'ancienne.

React a besoin d'une nouvelle référence

Les objets et tableaux d'état doivent être remplacés

immutable-update.tsx

```tsx
// à éviter
vehicles[0].status = "maintenance"
setVehicles(vehicles)

// à privilégier
setVehicles((current) =>
  current.map((vehicle) =>
    vehicle.id === vehicleId
      ? { ...vehicle, status: "maintenance" }
      : vehicle
  )
)
```

L'interface dépend de l'état actuel

Les conditions doivent rester lisibles et proches du rendu concerné

conditional-render.tsx

```tsx
{vehicles.length === 0 ? (
  <EmptyState />
) : (
  <FleetGrid vehicles={vehicles} />
)}

{hasWarning && <WarningBanner />}
```

Une key stable permet à React de suivre chaque véhicule

La position dans le tableau n'est pas une identité métier

list-keys.tsx

```tsx
// à privilégier
vehicles.map((vehicle) => (
  <VehicleCard
    key={vehicle.id}
    vehicle={vehicle}
  />
))

// à éviter
key={index}
```

TP 3 — Ground or Ready?

40 minutes · Rendez le registre de flotte interactif

1. Filtrer par statut
2. Afficher le nombre de résultats
3. Passer un véhicule en maintenance
4. Restaurer son statut `ready`
5. Afficher un état vide
6. Conserver les mises à jour immuables

- état typé
- clé stable
- aucune mutation
- aucune donnée filtrée enregistrée séparément

SÉQUENCE 4 · INCIDENT

Incident — tous les Titan sont immobilisés

Une inspection urgente concerne une catégorie entière

GROUND ALL TITAN VEHICLES

- seuls les Titan deviennent `grounded`
- les autres véhicules restent inchangés
- le compteur se met à jour
- le filtre continue de fonctionner

L'état minimal produit une interface plus fiable

Les résultats filtrés peuvent être calculés pendant le rendu

derived-filter.tsx

```tsx
// à éviter
const [vehicles, setVehicles] = useState(...)
const [filteredVehicles, setFilteredVehicles] =
  useState(...)

// à privilégier
const filteredVehicles = vehicles.filter(...)
```

Quelle donnée constitue la source de vérité ?

SÉQUENCE 5 · ÉVÉNEMENTS

04 — Réagir aux opérateurs

Chaque interaction transmet une information à React

TP 4 — Fleet Search

Les événements React possèdent leurs propres types

Le type dépend de l'élément qui déclenche l'interaction

event-types.tsx

```tsx
function handleSearch(
  event: React.ChangeEvent<HTMLInputElement>
) {
  setSearch(event.target.value)
}

// formulaire : React.FormEvent<HTMLFormElement>
// bouton     : React.MouseEvent<HTMLButtonElement>
```

React devient la source de vérité du champ

La valeur affichée correspond toujours à l'état

controlled-input.tsx

```tsx
const [search, setSearch] = useState("")

<input
  value={search}
  onChange={(event) =>
    setSearch(event.target.value)
  }
/>
```

Saisie → événement → setter → rendu → nouvelle valeur

La liste visible est une donnée dérivée

Recherche, statut et tri s'appliquent à la même source

visible-vehicles.tsx

```tsx
const visibleVehicles = vehicles
  .filter(matchesSearch)
  .filter(matchesStatus)
  .sort(compareVehicles)
```

Ne stockez pas une information que vous pouvez calculer à partir de l'état existant.

TP 4 — Fleet Search

30 minutes · Ajoutez recherche et tri

- recherche insensible à la casse
- filtre par statut
- tri par nom
- tri par disponibilité
- compteur de résultats
- bouton de réinitialisation

État vide — Aucun véhicule ne correspond à ces critères.

Bonus : mettre en évidence le texte recherché dans le nom du véhicule.

SÉQUENCE 5 · DÉBRIEF

Un bon filtre n'a pas besoin de useEffect

Les valeurs dérivées sont recalculées pendant le rendu

- source de vérité : `vehicles` + `search` + `selectedStatus` + `sortOrder`
- résultat dérivé : `visibleVehicles`

Quelles incohérences seraient possibles si les deux listes étaient stockées dans l'état ?

SÉQUENCE 6 · TAILWIND

05 — Habiller le cockpit

Une console technique doit rester lisible dans toutes les conditions

TP 5 — Mission Control Skin

Tailwind compose des utilitaires directement dans le JSX

Chaque classe applique une responsabilité visuelle limitée

tailwind-card.tsx

```tsx
<article
  className="
    rounded-xl
    border border-slate-700
    bg-slate-900
    p-6
  "
>
```

Les classes restent lisibles lorsqu'elles suivent un ordre cohérent.

Commencez par le plus petit écran

Les variantes ajoutent progressivement de l'espace et des colonnes

responsive-grid.tsx

```tsx
<div
  className="
    grid grid-cols-1 gap-4
    md:grid-cols-2
    xl:grid-cols-3
  "
>
```

Mobile → 1 colonne · tablette → 2 colonnes · bureau → 3 colonnes

Le mode nuit doit rester lisible

{ "Changer le fond ne suffit pas": "chaque état doit conserver son contraste" }

dark-mode.tsx

```
className="
  bg-white text-slate-950
  dark:bg-slate-950 dark:text-slate-100
"
```

- textes secondaires
- bordures
- statuts

La couleur doit transmettre une information

Chaque statut possède un traitement cohérent

READY | ASSIGNED | MAINTENANCE | GROUNDED

- ready – cyan ou vert
- assigned – bleu
- maintenance – orange
- grounded – rouge

Ne jamais transmettre un statut uniquement par la couleur : ajouter un libellé, une icône, une forme ou un motif.

TP 5 — Mission Control Skin

46 minutes · Donnez une identité au registre de flotte

1. une grille responsive
2. une hiérarchie visuelle claire
3. des statuts cohérents
4. un champ de recherche lisible
5. des états de focus visibles
6. un premier dark mode
7. un état vide soigné

L'interface doit rester compréhensible sans les couleurs.

AstraForge Design System — version 0.1

Une console spatiale sobre, technique et lisible

- fond — bleu nuit / presque noir
- texte — blanc froid
- primaire — cyan électrique
- alerte — orange
- critique — rouge
- surfaces — bordures fines, contraste modéré

Peu d'effets décoratifs, grands chiffres, labels courts, espace suffisant — pas de néon sur tous les éléments.

Trois viewports, une même mission

Le registre doit rester utilisable à chaque largeur

- 390 px — mobile
- 768 px — tablette
- 1440 px — bureau
- absence de scroll horizontal
- titres non coupés
- champs utilisables
- boutons accessibles
- métriques lisibles
- cartes cohérentes

Le Flight Deck est opérationnel si…

La qualité fonctionnelle et la qualité visuelle sont vérifiables

- composants à responsabilité claire
- props et événements typés
- clés stables
- mises à jour immuables
- recherche et filtres fonctionnels
- aucun état dérivé dupliqué
- interface responsive
- dark mode amorcé
- build sans erreur
- tag Git `session-02`

SÉQUENCE 7 · CLÔTURE

Flight Deck Review

Testez le cockpit d'un autre groupe

1. Trouvez-vous immédiatement un véhicule ?
2. Comprenez-vous chaque statut ?
3. La recherche est-elle prévisible ?
4. L'état vide est-il traité ?
5. Le mobile reste-t-il utilisable ?
6. Pouvez-vous naviguer au clavier ?

Feedback : continuer · arrêter · commencer

SÉQUENCE 7 · CLÔTURE

La flotte est maintenant interactive

Vos modèles TypeScript sont devenus une véritable interface React

- créer un arbre de composants
- définir des props typées
- composer avec `children`
- gérer un état local
- mettre à jour un tableau sans mutation
- typer les événements
- calculer un état dérivé
- construire une interface responsive

SÉQUENCE 7 · TICKET DE SORTIE

Avant de quitter le Flight Deck

Répondez individuellement en trois minutes

1. Quand créer un nouveau composant ?
2. Quelle différence entre prop et état ?
3. Pourquoi ne faut-il pas muter un tableau d'état ?
4. Pourquoi éviter de stocker `filteredVehicles` ?
5. Quand utiliseriez-vous `children` ?
6. Quelle partie de votre interface reste fragile ?

SÉQUENCE 7 · SUITE

Jour 3 — Router Flight Plan

Le cockpit va devenir une application multipage

- React Router v6
- layouts
- navigation
- routes dynamiques
- fiche détaillée d'un véhicule
- fiche mission
- page 404 Lost in Space

La flotte est visible. Demain, chaque module obtiendra sa propre destination.